Real-Time Facial Landmark Detection using MediaPipe

1 Overview

This project implements a real-time facial landmark detection system using MediaPipe Face Mesh. The system captures webcam input, extracts 468 facial landmarks, groups them into meaningful facial regions, and visualizes them using custom rendering.

2 Key Concepts Understood

Computer Vision: Computer Vision involves enabling machines to interpret visual data such as images and videos.here, it is applied through real-time video processing, facial landmark detection, and geometric representation of facial features.

Face Detection vs Landmark Detection: Face detection identifies the presence and location of a face (bounding box), whereas landmark detection estimates detailed geometric keypoints of the face.

Facial Landmarks: Each face is represented by 468 points with normalized (x, y, z) coordinates. In this project, only (x, y) coordinates are used since the goal is 2D visualization.

Face Mesh: Face Mesh provides dense landmark coverage, enabling fine-grained representation of facial structure such as eyes, lips, and contours.

3 Implementation Details

Pipeline:

1. Capture frames using OpenCV
2. Convert BGR to RGB
3. Convert RGB image to MediaPipe image format
4. Run MediaPipe Face Landmarker
5. Extract landmarks
6. Convert to NumPy array of shape $(F, 468, 2)$
7. Apply smoothing
8. Group landmarks into regions
9. Draw contours manually

Landmark Representation: All landmarks are converted into a NumPy array $(F, 468, 2)$, where F is the number of detected faces. A maximum of 2 faces is set using MediaPipe configuration. This structured format enables efficient processing and future compatibility with machine learning models.

Region Grouping: Instead of using all points blindly, landmarks are grouped into:

- Eyes
- Iris
- Eyebrows
- Lips
- Nose
- Face Outline

4 Additional Work and Improvements

1. Custom Visualization: Instead of using MediaPipe drawing utilities, all rendering is implemented manually using connection pairs.

2. Temporal Smoothing: A major issue observed was jitter in landmarks due to frame-to-frame variation. This was addressed using exponential smoothing:

$$\text{new} = \alpha \cdot \text{current} + (1 - \alpha) \cdot \text{previous}$$

This significantly improved visual stability.

3. Vectorized Processing: Smoothing was implemented using NumPy vectorization rather than loops, aligning the design with tensor-based machine learning pipelines.

4. Modular Design: The system is split into:

- `main.py` – pipeline control
- `face_mesh.py` – visualization
- `smooth.py` – stabilization

This improves readability and extensibility.

5 Experiments and Observations

- Landmarks fluctuate even when the face is stationary due to model noise.
- Lower smoothing factor reduces jitter but may introduce slight lag.
- The system works robustly across different lighting and angles.

6 Design Insights

- MediaPipe inference dominates computation time; smoothing is negligible.
- Vectorized operations improve code clarity and align with our future ML tasks.

7 Conclusion

This project goes beyond basic landmark extraction by introducing structured data representation, modular design, and temporal smoothing. It can be used as a strong foundation for later realtime ML tasks.